

Φάση 1 – Κλασική ανάκτηση

Θα φτιάξουμε ένα κλασικό σύστημα ανάκτησης πληροφορίας στην συλλογή IR2025 αξιοποιώντας την μηχανή αναζήτησης Elasticsearch.

Θα χρησιμοποιήσουμε την Elasticsearch οπότε αρχικά την κατεβάζουμε και μετά εκτελούμε την εντολή `>elasticsearch.bat` στον φάκελο `bin` για να ξεκινήσει, ενεργοποιώντας ένα node και ξεκινώντας την υπηρεσία.

```
C:\Users\evdok\elasticsearch-9.2.3-windows-x86_64\elasticsearch-9.2.3\bin>elasticsearch.bat
warning: ignoring JAVA_HOME=C:\Program Files\Java\jdk-17.0.2; using bundled JDK
[2026-01-07T22:54:44,860][INFO ][o.e.b.Elasticsearch      ] [LAPTOP-UDLON1P4] version[9.2.3], pid[23424],
build[zip/d8972a71dbbd64ff17f2f4dba9ca2c3fe09fb100/2025-12-16T10:09:08.849001802Z], OS[Windows 10/10.0/amd
64], JVM[Oracle Corporation/OpenJDK 64-Bit Server VM/25.0.1/25.0.1+8-27]
[2026-01-07T22:54:44,877][INFO ][o.e.b.Elasticsearch      ] [LAPTOP-UDLON1P4] JVM home [C:\Users\evdok\ela
sticsearch-9.2.3-windows-x86_64\elasticsearch-9.2.3\jdk], using bundled JDK [true]
[2026-01-07T22:54:44,879][INFO ][o.e.b.Elasticsearch      ] [LAPTOP-UDLON1P4] JVM arguments [-Des.networka
ddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Djava.awt.he
adless=true, -Dfile.encoding=UTF-8, -Djna.nosys=true, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUnsafe=
true, -Dio.netty.noKeySetOptimization=true, -Dio.netty.recycler.maxCapacityPerThread=0, --add-opens=org.ap
ache.lucene.core/org.apache.lucene.codecs.lucene99=org.elasticsearch.server, --add-opens=org.apache.lucene
.core/org.apache.lucene.codecs.hnsw=org.elasticsearch.server, --add-opens=org.apache.lucene.core/org.apach
```

Έχουμε απενεργοποιήσει το security layer της Elasticsearch για να μην απαιτείται αυθεντικοποίηση με username και password αφού αφορά τοπική ανάπτυξη και δεν θα δημιουργήσει πρόβλημα.

```
# Enable security features
xpack.security.enabled: false ##

xpack.security.enrollment.enabled: true

# Enable encryption for HTTP API client connections
xpack.security.http.ssl:
  enabled: true
  keystore.path: certs/http.p12

# Enable encryption and mutual authentication between nodes
xpack.security.transport.ssl:
  enabled: false ##
  verification_mode: certificate
  keystore.path: certs/transport.p12
  truststore.path: certs/transport.p12

# Create a new cluster with the current node only
# Additional nodes can still join the cluster later
cluster.initial_master_nodes: ["LAPTOP-UDLON1P4"]
```

Επιβεβαιώνουμε την σύνδεση με την Elasticsearch εκτελώντας την εντολή `!curl http://localhost:9200/`

```
[4]: !curl http://localhost:9200/
{
  "name" : "LAPTOP-UDLON1P4",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "6eRXGDVvKRza-7IQXW6AsSg",
  "version" : {
    "number" : "9.2.3",
    "build_flavor" : "default",
    "build_type" : "zip",
    "build_hash" : "d8972a71dbbd64ff17f2f4dba9ca2c3fe09fb100",
    "build_date" : "2025-12-16T10:09:08.849001802Z",
    "build_snapshot" : false,
    "lucene_version" : "10.3.2",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

1. Προεπεξεργαστείτε τη συλλογή κειμένων IR2025 (αρχείο `documents.csv`) προκειμένου να είναι σε κατάλληλη μορφή για να χρησιμοποιηθεί από τη μηχανή αναζήτησης Elasticsearch.

Αρχίζοντας την διαδικασία της προεπεξεργασίας βλέπουμε ότι η συλλογή κειμένων IR2025 βρίσκεται σε αρχείο csv οπότε εξάγουμε τα δεδομένα και φτιάχνουμε ένα jsonl αρχείο με αυτά ώστε να είναι συμβατό με την Elasticsearch. Το αρχείο θα έχει εγγραφές της μορφής {"doc_id": 193157, "text": "Support towards the .."}, όπου doc_id το μοναδικό αναγνωριστικό κάθε κειμένου από τα 18.316 και text το περιεχόμενο του κειμένου.

	ID	Text
1	193157	IC in its implementation, is explained below"
2	193158	ementation of the ERC under Horizon 2020.
3	193159	/ in the Consortium activities are envisaged.
4	193160	loping and operating the proposed services.
5	193161	Copernicus Service infrastructure in Europe.
6	193162	re stock assessment, and seabed mapping.
7	193163	ovative products that reflect societal needs.



```
root [] 18316 items
  0
    doc_id 193157
    text "Support towards the Europe PMC initiative-Contribution for 2
of its Open Access strategy for projects funded in the Life Science
SUP-2013) which has allowed the ERC to offer the benefits of Eurc
st the ERC in its implementation, is explained below"
  1
    doc_id 193158
    text "Support to the Vice-Presidents of the ERC Scientific Council 20
Council (ERC ScC) to achieve key milestones and deliverables of th
n 2020 programme, the Commission has established the European
of the key structural components of the ERC is the Scientific Coun
C independently establishes and oversees the ERC's scientific man
```

2. Δημιουργήστε ένα ευρετήριο από τη συλλογή χρησιμοποιώντας τη μηχανή αναζήτησης Elasticsearch. Επιλέξτε κατάλληλο Analyzer και συνάρτηση ομοιότητας (πχ. την BM25Similarity). Φτιάξτε τη δομή του ευρετηρίου σας όπως κρίνετε.

Φτιάχνουμε το ευρετήριο "my_idx" με settings ως εξής:

- Επιλέγουμε 1 shard αφού η συλλογή δεν είναι ογκώδης και τα κείμενα δεν είναι πολλά ούτε και μεγάλα.
- Για συνάρτηση ομοιότητας θα χρησιμοποιήσουμε την προεπιλεγμένη BM25Similarity, με παραμέτρους k1 ελέγχοντας την επίδραση της συχνότητα όρου (tf) και b ελέγχοντας την κανονικοποίηση προς το μήκος κειμένου.
- Για το tokenization κατά την ευρετηρίαση και την αναζήτηση θα επιλέξουμε τον English Analyzer αφού τα κείμενα είναι στα αγγλικά και είναι σχεδιασμένος να κάνει αποκοπή καταλήξεων (stemming), αφαίρεση των τετριμμένων λέξεων (stopword removal) και σημείων στίξης (punctuation removal) καθώς και πεζοποιεί τις λέξεις (lowercasing) ώστε να αυξήσει την συνάφεια και να επιστρέψει περισσότερα κατάλληλα κείμενα κατά την αναζήτηση. Έτσι μειώνει τις γλωσσικές παραλλαγές και επιστρέφει συναφή κείμενα ακόμα και αν το ερώτημα διατυπώνεται διαφορετικά από το ακριβές περιεχόμενο του κειμένου.
- Για το mapping φτιάχνουμε πεδία (fields) doc_id και text όπως υπάρχουν και στις εγγραφές του jsonl αρχείου.

```
"mappings": {
  "properties": {
    "doc_id": {"type": "integer"},
    "text": {"type": "text", "similarity": "my_bm25"}
  }
}

# create an index with the configuration above
client.indices.create(index='my_idx', body=my_mapping)

ObjectApiResponse({'acknowledged': True, 'shards_acknowledged': True, 'index': 'my_idx'})
```

Επιβεβαιώνουμε την δημιουργία του ευρετηρίου με τα configurations.

```
exists = client.indices.exists(index="my_idx")
print("Index exists: ", exists)

mapp= client.indices.get_mapping(index="my_idx")
print(mapp["my_idx"]["mappings"])

Index exists: True
{'properties': {'doc_id': {'type': 'integer'}, 'text': {'type': 'text', 'analyzer': 'default', 'similarity': 'my_bm25'}}}
```

Κάνουμε μαζική εισαγωγή (bulk insert) των κειμένων στο ευρετήριο χρησιμοποιώντας τις bulk helper συναρτήσεις της Elasticsearch αφού είναι πιο αποδοτική και γρήγορη σε σχέση με εισαγωγή μεμονωμένων κειμένων.

Ελέγχουμε αν η εισαγωγή έγινε σωστά εκτελώντας match-all query.

```
print("Done indexing documents into `my_idx` index!")
Done indexing documents into `my_idx` index!

[20]: search_results = client.search(index="my_movies", query={"match_all": {}})

# Print search results
pretty_search_response(search_results)

ID: k2_Y1JsBS7-TIK4up_cv
Score: 1.0
Doc ID: 204380
Text: A disruptive innovation for the minimisation of railway maintenance costs: Rail infrastructure (from public funds) in repairing railway track damages caused by the poor conditions of train wheels or the conditions of the train wheels (WID or Wheel Impact Detectors) to prevent/ minimise the damage accurately the occurrence of the failure or indicate to the train operating companies which maintain the optimum moment. AMINSA have developed DTD System, an innovative system capable of detecting wheel deterioration, and using this information to design a predictive maintenance plan. The added value of I - Its superior accuracy thanks to the new filtering process carried out by its innovative analysis
```

Εκτελούμε ένα παράδειγμα αναζήτησης για να δούμε πως δουλεύει. Παρατηρούμε ότι το πρώτο σε κατάταξη κείμενο περιέχει 3 φορές τον όρο neurostimulation (token: neurostimul)

```
search_results = client.search(
    index="my_idx",
    size=5,
    query = {"match": {"text": "neurostimulation"}},
)

# Print search results
pretty_search_response(search_results)

ID: 9xUonpsBUOuBdJ8z9aeE
Score: 12.3726845
Doc ID: 195939
Text: Understanding Emotional Actions: Emotions shape our personal lives and society at large. In Europe emotional disorders affect one in five people; it is the most chronic and second to most disabling health condition. The stakes to better understand our emotional behaviour are high; however there is still a lot to gain. Traditionally, emotions are thought to guide our behaviour either through fast instincts or deliberately selected actions. Current explanations of emotional health and disease frequently focus on one of these single components in isolation (or regard them as exclusive antagonists), but likely underestimate our emotional behaviour's complexity and diversity. Here I will investigate a complementary account: I suggest that in humans optimal emotional behaviour arises not from these instinctual and deliberate action selection systems individually, but from their interaction, mediated in our brain by a likely uniquely human prefrontal control system. First, using an innovative task, behavioural profiling, and formal computational modelling I will determine the influence and interaction of these systems in emotional behaviour. Second, I will use concurrent neuroimaging (fMRI) and neurostimulation (tDCS), geared at the prefrontal control system and its connectivity with amygdala and parietal cortex, to investigate their causal mechanisms during emotional situations. This ambitious project combines my extensive experience in emotional behaviour with the world-leading expertise of University College London in computational neuroimaging and neurostimulation. These investigations are deliberately designed to both further our understanding of emotional behaviour and to initiate a path for improved treatment of emotional disorders in Europe. The computational modelling approach will elucidate hidden mechanisms and promote personalized diagnosis, while the modulatory neurostimulation approach will provide new avenues for targeted treatment ready to be developed in collaboration with the clinic.
```

Στην δεύτερη θέση και με λίγο χαμηλότερο σκορ είναι ένα κείμενο που περιέχει και αυτό 3 φορές τον όρο του ερωτήματος. Όπως θα ελέγξουμε, αυτή η διαφορά στο σκορ οφείλεται στο μήκος των κειμένων μετά το tokenization. Συγκεκριμένα στο πρώτο κείμενο εμφανίζεται το token “neurostimul” 3 φορές στα 184 συνολικά tokens, ενώ στο δεύτερο εμφανίζεται 3 φορές στα 200 tokens συνολικά. Αυτό εξηγεί την κατάταξη.

```
ID: ZRUonpsBUOuBdJ8zJ75o
Score: 12.153968
Doc ID: 204389
Text: Ultra-small implanted neurostimulation device for the treatment of Obstructive Sleep Apnea: Obstructive Sleep Apnea is the world's most common sleep disorder and affects 100m people worldwide. It is particularly prevalent among elderly and obese cohorts and is a lifelong disease, with acute and chronic health impacts. It is linked to increased cardiovascular morbidity and mortality, and associated with increased risk of heart attack, cardiac arrhythmia, stroke, daytime sleepiness, depression and poor neurocognitive performance. Estimated annual economic costs of the disease are €60-150b. Established therapeutic options are uncomfortable with unpleasant side effects and/or ineffective; they can also require invasive surgery. Compliance is a major problem with the first line treatment, CPAP: 50% of users refuse it outright or fail to continue use after 1 year. Nyxoah's disruptive neurostimulation device responds directly to these needs and drawbacks. Central to the system is a tiny device implanted under the chin that - compared to other neurostimulator implantations - is smaller, less invasive and has lower risk of infection. The device is powered wirelessly by an activation chip, which is attached to a disposable smart patch and stuck under the chin before sleep. No mask, no noisy machine, no unpleasant side effects. The chip has a closed feedback loop with the implant, which detects tissue resistance and tells the chip to adjust the pulse to keep constant pressure on the nerve, to avoid burnout; it also communicates wirelessly with an app, which displays useful information (synced with the cloud) for use by patients, physicians and social security agencies; this is a key development to allow treatment compliance monitoring and therefore for reimbursement. During the Phase 1 project, the company, founded in 2009, will assess scale-up industrialization plan, product development according to regulatory standards and clinical trials to be deployed in Europe to show the system efficacy on patient outcome and public healthcare costs.
```

```
## dl (document length after tokenization) = 184 for document 195939 (1st place)
```

```
client.explain(  
  index="my_idx",  
  id="9xUonpsBUOuBdJ8z9aeE",  
  body={  
    "query": {  
      "match": {  
        "text": "neurostimulation"  
      }  
    }  
  }  
)
```

```
ObjectApiResponse({'_index': 'my_idx', '_id': '9xUonpsBUOuBdJ8z9aeE', 'matched': True, 'explanation': {'value': 12.3726845, 'description': 'weight(text:neurostimul in 267) [PerFieldSimilarity], result of:', 'details': [{'value': 12.3726845, 'description': 'score(freq=3.0), computed as boost * idf * tf from:', 'details': [{'value': 2.2, 'description': 'boost', 'details': []}, {'value': 7.800682, 'description': 'idf, computed as log(1 + (N - n + 0.5) / (n + 0.5)) from:', 'details': [{'value': 7, 'description': 'n, number of documents containing term', 'details': []}, {'value': 18316, 'description': 'N, total number of documents with field', 'details': []}], {'value': 0.72095585, 'description': 'tf, computed as freq / (freq + k1 * (1 - b + b * dl / avgdl)) from:', 'details': [{'value': 3.0, 'description': 'freq, occurrences of term within document', 'details': []}, {'value': 1.2, 'description': 'k1, term saturation parameter', 'details': []}, {'value': 0.75, 'description': 'b, length normalization parameter', 'details': []}, {'value': 184.0, 'description': 'dl, length of field (approximate)', 'details': []}, {'value': 192.30269, 'description': 'avgdl, average length of field', 'details': []}]}}]}
```

```
## dl (document length after tokenization) = 200 for document 204389 (2nd place)
```

```
client.explain(  
  index="my_idx",  
  id="ZRUpnpsBUOuBdJ8zJ75o",  
  body={  
    "query": {  
      "match": {  
        "text": "neurostimulation"  
      }  
    }  
  }  
)
```

```
ObjectApiResponse({'_index': 'my_idx', '_id': 'ZRUpnpsBUOuBdJ8zJ75o', 'matched': True, 'explanation': {'value': 12.153968, 'description': 'weight(text:neurostimul in 2988) [PerFieldSimilarity], result of:', 'details': [{'value': 12.153968, 'description': 'score(freq=3.0), computed as boost * idf * tf from:', 'details': [{'value': 2.2, 'description': 'boost', 'details': []}, {'value': 7.800682, 'description': 'idf, computed as log(1 + (N - n + 0.5) / (n + 0.5)) from:', 'details': [{'value': 7, 'description': 'n, number of documents containing term', 'details': []}, {'value': 18316, 'description': 'N, total number of documents with field', 'details': []}], {'value': 0.7082112, 'description': 'tf, computed as freq / (freq + k1 * (1 - b + b * dl / avgdl)) from:', 'details': [{'value': 3.0, 'description': 'freq, occurrences of term within document', 'details': []}, {'value': 1.2, 'description': 'k1, term saturation parameter', 'details': []}, {'value': 0.75, 'description': 'b, length normalization parameter', 'details': []}, {'value': 200.0, 'description': 'dl, length of field (approximate)', 'details': []}, {'value': 192.30269, 'description': 'avgdl, average length of field', 'details': []}]}}]}
```

Βγάζουμε το συμπέρασμα λοιπόν πως παρότι τα δύο κείμενα εμφανίζονται παρόμοια σε μήκος και την συχνότητα του όρου ερωτήματος, υπάρχει διαφορά στο σκορ συνάφειας που οφείλεται στην συχνότητα των tokens μετά την ανάλυση, στην κανονικοποίηση του μήκους κειμένου και στον κορεσμό της συχνότητας όρων που εφαρμόζει η συνάρτηση ομοιότητας του BM25.

3. Εκτελέστε τα ερωτήματα (αρχείο queries.csv) πάνω στο ευρετήριο και συλλέξτε τις απαντήσεις της μηχανής, τα k πρώτα ανακτηθέντα κείμενα, για k = 20, 30, 50.

Αρχικά θα τα εξάγουμε τα ερωτήματα από το αρχείο queries.csv σε δομή pandas dataframe, όπου κάθε γραμμή αντιστοιχεί σε ένα ερώτημα με το μοναδικό αναγνωριστικό και το περιεχόμενό του.

Θα φτιάξουμε την μέθοδο `queries_run` η οποία εκτελεί όλα τα ερωτήματα πάνω στο ευρετήριο `my_idx` της `elasticsearch` και επιστρέφει κάθε φορά τα πρώτα k κείμενα, ανάλογα με το σκορ ομοιότητας, σε μορφή κατάλληλη για το εργαλείο `trec_eval` που θα χρησιμοποιήσουμε στην συνέχεια για την αξιολόγηση των αποτελεσμάτων.

Συγκεκριμένα κάνει τα εξής:

- Αρχικοποιεί μια λίστα που θα περιέχει τα αποτελέσματα όλων των ερωτημάτων, την οποία στο τέλος θα επιστρέψει.
- Με επαναληπτικό τρόπο και για κάθε ερώτημα εκτελεί free text match query στο πεδίο text των κειμένων και επιστρέφει τα k πιο συναφή κείμενα σύμφωνα με τον BM25 που χρησιμοποιήθηκε στο ευρετήριο. Σε αυτή την φάση γίνεται το tokenization του ερωτήματος ώστε να γίνει η αναζήτηση και η ταξινόμηση με το BM25Similarity όπως έχει οριστεί στο ευρετήριο.
- Σε εσωτερική επανάληψη και για κάθε ανακτημένο κείμενο του αποτελέσματος της αναζήτησης, αποθηκεύει το `doc_id` και το `similarity score` του κειμένου καθώς και το `query_id` του ερωτήματος και την θέση κατάταξης rank σε μορφή κατάλληλη για την αξιολόγηση που θα ακολουθήσει.


```

results_20 = queries_run(client, queries, k=20)
print(results_20)
results_30 = queries_run(client, queries, k=30)
results_50 = queries_run(client, queries, k=50)

```

```

[{'query_id': 'Q01', 'doc_id': 193378, 'rank': 0, 'score': 764.02673}, {'query_id': 'Q01', 'doc_id': 193373, 'rank': 0, 'score': 273.72284}, {'query_id': 'Q01', 'doc_id': 205685, 'rank': 0, 'score': 244.1519}, {'query_id': 'Q01', 'doc_id': 193715, 'rank': 0, 'score': 242.06233}, {'query_id': 'Q01', 'doc_id': 206230, 'rank': 0, 'score': 222.9547}, {'query_id': 'Q01', 'doc_id': 193353, 'rank': 0, 'score': 222.53535}, {'query_id': 'Q01', 'doc_id': 193375, 'rank': 0, 'score': 220.6753}, {'query_id': 'Q01', 'doc_id': 193386, 'rank': 0, 'score': 216.5555}, {'query_id': 'Q01', 'doc_id': 211346, 'rank': 0, 'score': 215.89311}, {'query_id': 'Q01', 'doc_id': 202703, 'rank': 0, 'score': 213.50832}, {'query_id': 'Q01', 'doc_id': 206228, 'rank': 0, 'score': 212.71326}, {'query_id': 'Q01', 'doc_id': 213250, 'rank': 0, 'score': 212.42722}, {'query_id': 'Q01', 'doc_id': 194660, 'rank': 0, 'score': 210.28812}, {'query_id': 'Q01', 'doc_id': 210137, 'rank': 0, 'score': 207.94757}, {'query_id': 'Q01', 'doc_id': 213278, 'rank': 0, 'score': 206.191}, {'query_id': 'Q01', 'doc_id': 205643, 'rank': 0, 'score': 206.60056}, {'query_id': 'Q01', 'doc_id': 211970, 'rank': 0, 'score': 204.75098}, {'query_id': 'Q01', 'doc_id': 211972, 'rank': 0, 'score': 199.05475}, {'query_id': 'Q01', 'doc_id': 206824, 'rank': 0, 'score': 197.84494}, {'query_id': 'Q01', 'doc_id': 193402, 'rank': 0, 'score': 197.28416}, {'query_id': 'Q02', 'doc_id': 213164, 'rank': 0, 'score': 520.1175}, {'query_id': 'Q02', 'doc_id': 210232, 'rank': 0, 'score': 202.63286}, {'query_id': 'Q02', 'doc_id': 194301, 'rank': 0, 'score': 155.4609}, {'query_id': 'Q02', 'doc_id': 19

```

Φτιάχνουμε την μέθοδο `file_results` όπου τα προηγούμενα αποτελέσματα θα αποθηκευτούν σε ένα txt αρχείο για να χρησιμοποιηθεί στο `trec_eval`.

Τα αρχεία θα έχουν αυτή την μορφή:

```

with open("trec_eval/run_bm25_k20.txt") as file:
    print(file.read())

```

```

Q01 Q0 193378 0 764.02673 bm25_k20
Q01 Q0 193373 0 273.72284 bm25_k20
Q01 Q0 205685 0 244.1519 bm25_k20
Q01 Q0 193715 0 242.06233 bm25_k20
Q01 Q0 206230 0 222.9547 bm25_k20
Q01 Q0 193353 0 222.53535 bm25_k20
Q01 Q0 193375 0 220.6753 bm25_k20
Q01 Q0 193386 0 216.5555 bm25_k20
Q01 Q0 211346 0 215.89311 bm25_k20
Q01 Q0 202703 0 213.50832 bm25_k20
Q01 Q0 206228 0 212.71326 bm25_k20
Q01 Q0 213250 0 212.42722 bm25_k20
Q01 Q0 194660 0 210.28812 bm25_k20
Q01 Q0 210137 0 207.94757 bm25_k20
Q01 Q0 213278 0 206.191 bm25_k20
Q01 Q0 205643 0 206.60056 bm25_k20
Q01 Q0 211970 0 204.75098 bm25_k20
Q01 Q0 211972 0 199.05475 bm25_k20
Q01 Q0 206824 0 197.84494 bm25_k20
Q01 Q0 193402 0 197.28416 bm25_k20
Q02 Q0 213164 0 520.1175 bm25_k20
Q02 Q0 210232 0 202.63286 bm25_k20
Q02 Q0 194301 0 155.4609 bm25_k20
Q02 Q0 194660 0 155.4609 bm25_k20

```

4. Αξιολογήστε τις απαντήσεις σας συγκρίνοντάς τις με τις σωστές απαντήσεις (αρχείο `qrels.txt`) χρησιμοποιώντας το εργαλείο αξιολόγησης `trec_eval` και τα μέτρα αξιολόγησης MAP (mean average precision) και $Precision@k$ (ακρίβεια στα k πρώτα ανακτηθέντα κείμενα) για $k = 5, 10, 15, 20$.

Μετακινούμε τα 3 αρχεία που δημιουργήθηκαν στον φάκελο με το εργαλείο `trec_eval`.

Αρχικά εκτελούμε την εντολή `>trec_eval -q qrels.txt run_bm25_k20.txt` για να πάρουμε τα αποτελέσματα της αξιολόγησης στα **k=20 κείμενα**.

Μετά `>trec_eval -q qrels.txt run_bm25_k30.txt` για **k=30 κείμενα** και `>trec_eval -q qrels.txt run_bm25_k50.txt` για **k=50 κείμενα**.

```

map      all      0.6895
gm_map   all      0.6832
Rprec    all      0.7130
bpref    all      0.8214
recip_rank all    0.9500
iprec_at_recall_0.00 all  0.9875
iprec_at_recall_0.10 all  0.9875
iprec_at_recall_0.20 all  0.9708
iprec_at_recall_0.30 all  0.9048
iprec_at_recall_0.40 all  0.8302
iprec_at_recall_0.50 all  0.8227
iprec_at_recall_0.60 all  0.7858
iprec_at_recall_0.70 all  0.7535
iprec_at_recall_0.80 all  0.5292
iprec_at_recall_0.90 all  0.0650
iprec_at_recall_1.00 all  0.0000
P_5      all      0.8800
P_10     all      0.7300
P_15     all      0.7000
P_20     all      0.6300
P_30     all      0.4200
P_100    all      0.1260
P_200    all      0.0630
P_500    all      0.0252
P_1000   all      0.0126

```

```

map      all      0.7471
gm_map   all      0.7410
Rprec    all      0.7130
bpref    all      0.9209
recip_rank all    0.9500
iprec_at_recall_0.00 all  0.9875
iprec_at_recall_0.10 all  0.9875
iprec_at_recall_0.20 all  0.9708
iprec_at_recall_0.30 all  0.9048
iprec_at_recall_0.40 all  0.8302
iprec_at_recall_0.50 all  0.8227
iprec_at_recall_0.60 all  0.7858
iprec_at_recall_0.70 all  0.7535
iprec_at_recall_0.80 all  0.6655
iprec_at_recall_0.90 all  0.4684
iprec_at_recall_1.00 all  0.1794
P_5      all      0.8800
P_10     all      0.7300
P_15     all      0.7000
P_20     all      0.6300
P_30     all      0.4700
P_100    all      0.1410
P_200    all      0.0705
P_500    all      0.0282
P_1000   all      0.0141

```

```

map      all      0.7701
gm_map   all      0.7634
Rprec    all      0.7130
bpref    all      0.9757
recip_rank all    0.9500
iprec_at_recall_0.00 all  0.9875
iprec_at_recall_0.10 all  0.9875
iprec_at_recall_0.20 all  0.9708
iprec_at_recall_0.30 all  0.9048
iprec_at_recall_0.40 all  0.8302
iprec_at_recall_0.50 all  0.8227
iprec_at_recall_0.60 all  0.7858
iprec_at_recall_0.70 all  0.7535
iprec_at_recall_0.80 all  0.6655
iprec_at_recall_0.90 all  0.5300
iprec_at_recall_1.00 all  0.4166
P_5      all      0.8800
P_10     all      0.7300
P_15     all      0.7000
P_20     all      0.6300
P_30     all      0.4700
P_100    all      0.1490
P_200    all      0.0745
P_500    all      0.0298
P_1000   all      0.0149

```

Εικόνα 1: $k=20$

Εικόνα 2: $k=30$

Εικόνα 3: $k=50$

Καταλήγουμε σε αυτόν τον πίνακα αποτελεσμάτων:

k	20	30	50
map	0.6895	0.7471	0.7701
P_5	0.8800	0.8800	0.8800
P_10	0.7300	0.7300	0.7300
P_15	0.7000	0.7000	0.7000
P_20	0.6300	0.6300	0.6300
P_30	0.4200	0.4700	0.4700
P_100	0.1260	0.1410	0.1490
P_200	0.0630	0.0705	0.0745
P_500	0.0252	0.0282	0.0298
P_1000	0.0126	0.0141	0.0149

Παρατηρούμε ότι όσο αυξάνεται το k αυξάνεται και το map (mean average precision). Αυτό σημαίνει ότι με την αύξηση του πλήθους των ανακτηθέντων κείμενων k γίνεται ανάκτηση όλο και πιο συναφών εγγράφων, άρα αυξάνεται και η μέση ακρίβεια. Υπολογίζοντας την αύξηση του map από τα 20 στα 30 κείμενα κατά 0.0576 (0.7471-0.6895) , και από τα 30 στα 50 κείμενα κατά 0.023 (0.7701-0.7471), μπορούμε να πούμε ότι το map βελτώνεται πιο πολύ από τα πρώτα 20 κείμενα στα 30, ενώ από τα 30 στα 50 η βελτίωση είναι μικρότερη.

Βλέπουμε ότι για σταθερό πλήθος ανακτηθέντων κειμένων η ακρίβεια (precision) μειώνεται όσο αυξάνεται το k , που είναι λογικό αφού μπαίνουν όλο και περισσότερα μη συναφή κείμενα στην ανάκτηση. Το Precision@ k για k μέχρι 20 κείμενα παραμένει σταθερό για τα πρώτα 20,30 και 50 ανακτηθέντα κείμενα, συμπεραίνοντας ότι το σύστημα ανακτά τα συναφή έγγραφα νωρίς και ότι η κατάταξη των ανακτηθέντων εγγράφων είναι σταθερή για τα 20 πρώτα.

Τελικά, μπορούμε να συμπεράνουμε ότι το σύστημα ανάκτησης που φτιάξαμε πετυχαίνει υψηλή ακρίβεια ,με MAP γύρω στο 70% στα πρώτα αποτελέσματα. Οι τιμές του Precision@5 μέχρι Precision@20 είναι σταθερές και ανεξάρτητες του πλήθους των ανακτηθέντων κειμένων, κάτι που δείχνει ότι η κατάταξη στα κορυφαία αποτελέσματα είναι σταθερή και άρα αξιόπιστη. Γενικά, η αύξηση που MAP αν και φθίνουσα υποδεικνύει ότι το σύστημα συνεχίζει να ανακτά συναφή κείμενα και σε χαμηλότερη κατάταξη.